In [ ]:
```
# This Python 3 environment comes with many helpful analytics libraries i
# It is defined by the kaggle/python Docker image: https://github.com/kag
# For example, here's several helpful packages to load

import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)

# Input data files are available in the read-only "../input/" directory
# For example, running this (by clicking run or pressing Shift+Enter) wil

import os
for dirname, _, filenames in os.walk('/kaggle/input'):
    for filename in filenames:
        print(os.path.join(dirname, filename))

# You can write up to 20GB to the current directory (/kaggle/working/) th
# You can also write temporary files to /kaggle/temp/, but they won't be
```

In [ ]:
```
# ─────────────────────────────────────────────────────────────────────────
# CELL 1: Install & Import Libraries
# ─────────────────────────────────────────────────────────────────────────

# Install required packages
!pip install -q transformers datasets sacrebleu sacremoses sentencepiece

# ── Standard Library ──────────────────────────────────────────────────────
import os
import re
import math
import warnings
warnings.filterwarnings('ignore')

# ── Numerical & Visualization ─────────────────────────────────────────────
import numpy as np
import matplotlib
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker

# ── PyTorch ───────────────────────────────────────────────────────────────
import torch
print(f"PyTorch version  : {torch.__version__}")
print(f"CUDA available   : {torch.cuda.is_available()}")
if torch.cuda.is_available():
    print(f"GPU device       : {torch.cuda.get_device_name(0)}")
    print(f"GPU memory       : {torch.cuda.get_device_properties(0).total

DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

# ── HuggingFace ───────────────────────────────────────────────────────────
import transformers
from transformers import (
    T5ForConditionalGeneration,
    T5Tokenizer,
    Seq2SeqTrainer,
    Seq2SeqTrainingArguments,
    DataCollatorForSeq2Seq,
    EarlyStoppingCallback,
    TrainerCallback,
```

```
)
from datasets import load_dataset
print(f"Transformers ver : {transformers.__version__}")

# ── NLP / Metrics ──────────────────────────────────────────────────
import nltk
nltk.download('punkt',     quiet=True)
nltk.download('punkt_tab', quiet=True)
from nltk.translate.bleu_score import corpus_bleu, SmoothingFunction
from nltk.tokenize import word_tokenize

# ── Reproducibility ────────────────────────────────────────────────
SEED = 42
torch.manual_seed(SEED)
np.random.seed(SEED)

print("\n✅ All libraries imported successfully.")
```

In [ ]:

```
# ──────────────────────────────────────────────────────────────────
# CELL 2: Load Dataset
# ──────────────────────────────────────────────────────────────────

from datasets import load_dataset

print("Loading dataset: eilamc14/wikilarge-clean ...")
dataset = load_dataset("eilamc14/wikilarge-clean")
print("\nDataset loaded successfully!")
print(dataset)

# ── Show available splits ──────────────────────────────────────────
print("\n" + "="*55)
print("AVAILABLE SPLITS")
print("="*55)
for split_name, split_data in dataset.items():
    print(f"  {split_name:12s}: {len(split_data):>7,} samples")
    print(f"                columns: {split_data.column_names}")

# ── Show a raw sample ──────────────────────────────────────────────
print("\n" + "="*55)
print("RAW SAMPLE (train[0])")
print("="*55)
sample = dataset['train'][0]
for k, v in sample.items():
    print(f"  [{k}]: {str(v)[:150]}")

# ── Auto-detect column names ───────────────────────────────────────
cols = dataset['train'].column_names
COMPLEX_COL = None
SIMPLE_COL  = None

for c in cols:
    cl = c.lower()
    if any(kw in cl for kw in ['complex', 'src', 'source', 'normal', 'ori
        COMPLEX_COL = c
    if any(kw in cl for kw in ['simple', 'tgt', 'target', 'simp']):
        SIMPLE_COL = c

# Fallback: first col = complex, second = simple
if COMPLEX_COL is None: COMPLEX_COL = cols[0]
if SIMPLE_COL  is None: SIMPLE_COL  = cols[1]
```

```
print(f"\nDetected → COMPLEX column : '{COMPLEX_COL}'")
print(f"           SIMPLE  column : '{SIMPLE_COL}'")

# ── Reduce size for faster training ──────────────────────────────
MAX_TRAIN = 30_000
MAX_VAL   =  3_000

train_dataset = dataset['train'].shuffle(seed=SEED)
if len(train_dataset) > MAX_TRAIN:
    train_dataset = train_dataset.select(range(MAX_TRAIN))

if 'validation' in dataset:
    val_dataset = dataset['validation']
elif 'test' in dataset:
    val_dataset = dataset['test']
else:
    split         = train_dataset.train_test_split(test_size=0.1, seed=SE
    train_dataset = split['train']
    val_dataset   = split['test']

if len(val_dataset) > MAX_VAL:
    val_dataset = val_dataset.select(range(MAX_VAL))

print(f"\nFinal split sizes:")
print(f"  Train      : {len(train_dataset):,}")
print(f"  Validation : {len(val_dataset):,}")
print("\n✅ Dataset loaded and split complete.")
```

In [ ]:

```
# ──────────────────────────────────────────────────────────────────
# CELL 3: Explore Dataset
# ──────────────────────────────────────────────────────────────────

import matplotlib.pyplot as plt
import numpy as np

print("="*65)
print("DATASET EXPLORATION")
print("="*65)

# ── Word-count distribution ──────────────────────────────────────
EXPLORE_N = 2000   # Sample size for fast exploration
explore_subset = train_dataset.select(range(min(EXPLORE_N, len(train_data

complex_lengths = [len(ex[COMPLEX_COL].split()) for ex in explore_subset]
simple_lengths  = [len(ex[SIMPLE_COL].split())  for ex in explore_subset]

print(f"\nWord Count Statistics (sample of {EXPLORE_N} train examples):")
print(f"{'Metric':<25} {'Complex':>10} {'Simple':>10}")
print("-" * 47)
print(f"{'Mean':<25} {np.mean(complex_lengths):>10.1f} {np.mean(simple_l
print(f"{'Median':<25} {np.median(complex_lengths):>10.1f} {np.median(si
print(f"{'Std Dev':<25} {np.std(complex_lengths):>10.1f} {np.std(simple_
print(f"{'Min':<25} {np.min(complex_lengths):>10.0f} {np.min(simple_leng
print(f"{'Max':<25} {np.max(complex_lengths):>10.0f} {np.max(simple_leng

# ── Print 5 example pairs ────────────────────────────────────────
print("\n" + "="*65)
print("EXAMPLE SENTENCE PAIRS")
print("="*65)
```

```
for i in range(5):
    ex = explore_subset[i]
    print(f"\n[Pair {i+1}]")
    print(f"  COMPLEX  : {ex[COMPLEX_COL][:120]}")
    print(f"  SIMPLE   : {ex[SIMPLE_COL][:120]}")

# ── Plot word-count distribution ──────────────────────────────────────
fig, axes = plt.subplots(1, 2, figsize=(14, 4))
fig.suptitle("Dataset Word Count Distribution", fontsize=14, fontweight='

for ax, lengths, label, color in zip(
    axes,
    [complex_lengths, simple_lengths],
    ['Complex Sentences', 'Simple Sentences'],
    ['#3a86ff', '#ff006e']
):
    ax.hist(lengths, bins=40, color=color, alpha=0.80, edgecolor='white')
    ax.axvline(np.mean(lengths), color='black', linestyle='--',
               linewidth=1.5, label=f'Mean = {np.mean(lengths):.1f}')
    ax.set_title(label, fontsize=12)
    ax.set_xlabel('Word Count', fontsize=11)
    ax.set_ylabel('Frequency',  fontsize=11)
    ax.legend(fontsize=10)
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)

plt.tight_layout()
plt.savefig("/kaggle/working/word_count_distribution.png", dpi=150, bbox_
plt.show()

# ── Length reduction insight ──────────────────────────────────────────
reductions = [(c - s) / c * 100 for c, s in zip(complex_lengths, simple_l
print(f"\nAverage word count reduction (complex → simple): {np.mean(reduc
print("\n✅ Dataset exploration complete.")
```

In [ ]:

```
# ──────────────────────────────────────────────────────────────────────
# CELL 3 (FIXED): Preprocessing & Tokenization
# Fix: Removed deprecated `tokenizer.as_target_tokenizer()`
#      Use `text_target=` parameter directly instead.
# ──────────────────────────────────────────────────────────────────────

PREFIX     = "simplify: "
MAX_INPUT  = 256
MAX_TARGET = 128

import re

def clean_text(text: str) -> str:
    if not isinstance(text, str):
        return ""
    return re.sub(r'\s+', ' ', text).strip()

def preprocess_batch(examples):
    """
    Tokenize a batch of (complex, simple) pairs.
    - Uses `text_target=` instead of the removed `as_target_tokenizer()`
    - Replaces pad token IDs in labels with -100 (ignored by cross-entrop
    """
    sources = [PREFIX + clean_text(s) for s in examples[COMPLEX_COL]]
    targets = [clean_text(s)          for s in examples[SIMPLE_COL]]
```

```
    # ── Single tokenizer call: source + target together ──────
    # `text_target` is the modern replacement for `as_target_tokenizer()`
    model_inputs = tokenizer(
        sources,
        text_target  = targets,        # <-- KEY FIX
        max_length   = MAX_INPUT,
        max_target_length = MAX_TARGET,
        truncation   = True,
        padding      = "max_length",
    )

    # Replace padding token id in labels with -100 so loss ignores them
    model_inputs["labels"] = [
        [(tok if tok != tokenizer.pad_token_id else -100) for tok in labe
        for label in model_inputs["labels"]
    ]

    return model_inputs


# ── Filter empty / very short samples ──────────────────────────
def is_valid(example):
    src = clean_text(example.get(COMPLEX_COL, ""))
    tgt = clean_text(example.get(SIMPLE_COL,  ""))
    return len(src.split()) >= 3 and len(tgt.split()) >= 3

print("Filtering invalid samples ...")
train_dataset_clean = train_dataset.filter(is_valid)
val_dataset_clean   = val_dataset.filter(is_valid)
print(f"After filtering → train: {len(train_dataset_clean):,}  val: {len(

# ── Apply tokenization ─────────────────────────────────────────
print("\nTokenizing train split ...")
tokenized_train = train_dataset_clean.map(
    preprocess_batch,
    batched        = True,
    batch_size     = 512,
    remove_columns = train_dataset_clean.column_names,
    desc           = "Tokenizing train",
)

print("Tokenizing validation split ...")
tokenized_val = val_dataset_clean.map(
    preprocess_batch,
    batched        = True,
    batch_size     = 512,
    remove_columns = val_dataset_clean.column_names,
    desc           = "Tokenizing validation",
)

tokenized_train.set_format(type='torch')
tokenized_val.set_format(type='torch')

print(f"\nTokenized train columns : {tokenized_train.column_names}")
print(f"Tokenized val   columns : {tokenized_val.column_names}")

sample = tokenized_train[0]
print(f"\nSample input_ids shape  : {sample['input_ids'].shape}")
```

```
print(f"Sample labels length    : {len(sample['labels'])}")
print("\n✅ Preprocessing complete.")
```

In [ ]:
```
# ──────────────────────────────────────────────────────────────
# CELL 4: Load T5-Base Model & Move to GPU
# ──────────────────────────────────────────────────────────────

# Clear GPU cache before loading model
torch.cuda.empty_cache()

print(f"Loading model: {MODEL_NAME} ...")
model = T5ForConditionalGeneration.from_pretrained(MODEL_NAME)

# Move model to GPU
model = model.to(DEVICE)

# ── Model Info ────────────────────────────────────────────────
total_params     = sum(p.numel() for p in model.parameters())
trainable_params = sum(p.numel() for p in model.parameters() if p.require

print(f"\nModel        : {MODEL_NAME}")
print(f"Device       : {next(model.parameters()).device}")
print(f"Total params : {total_params/1e6:.1f}M")
print(f"Trainable    : {trainable_params/1e6:.1f}M")

if torch.cuda.is_available():
    allocated = torch.cuda.memory_allocated() / 1e9
    reserved  = torch.cuda.memory_reserved()  / 1e9
    print(f"GPU memory   : allocated={allocated:.2f}GB  reserved={reserve

print("\n✅ Model loaded and on GPU.")
```

In [ ]:
```
# ──────────────────────────────────────────────────────────────
# CELL 5 (FIXED v3): Training Setup
# Fixes applied:
#   1. warmup_ratio      → warmup_steps (calculated manually)
#   2. logging_dir       → removed (use TENSORBOARD_LOGGING_DIR env var i
#   3. Seq2SeqTrainer    → tokenizer kwarg removed; use processing_class
# ──────────────────────────────────────────────────────────────

import os
import math
import torch
torch.cuda.empty_cache()

from transformers import (
    DataCollatorForSeq2Seq,
    Seq2SeqTrainer,
    Seq2SeqTrainingArguments,
    EarlyStoppingCallback,
    TrainerCallback,
)

OUTPUT_DIR = "/kaggle/working/t5-text-simplification"
os.makedirs(OUTPUT_DIR, exist_ok=True)

# ── Calculate warmup_steps manually (replaces warmup_ratio) ──
EPOCHS         = 4
TRAIN_SAMPLES  = len(tokenized_train)
```

```
BATCH_SIZE     = 8
GRAD_ACCUM     = 2
WARMUP_RATIO   = 0.05

steps_per_epoch  = math.ceil(TRAIN_SAMPLES / (BATCH_SIZE * GRAD_ACCUM))
total_steps      = steps_per_epoch * EPOCHS
warmup_steps_val = max(1, int(total_steps * WARMUP_RATIO))

print(f"Steps per epoch : {steps_per_epoch}")
print(f"Total steps     : {total_steps}")
print(f"Warmup steps    : {warmup_steps_val}")

# ── Data Collator ──────────────────────────────────────────
data_collator = DataCollatorForSeq2Seq(
    tokenizer          = tokenizer,
    model              = model,
    padding            = True,
    pad_to_multiple_of = 8,
)

# ── Training Arguments ─────────────────────────────────────
training_args = Seq2SeqTrainingArguments(
    output_dir                  = OUTPUT_DIR,

    # Epochs & Batch
    num_train_epochs            = EPOCHS,
    per_device_train_batch_size = BATCH_SIZE,
    per_device_eval_batch_size  = 16,
    gradient_accumulation_steps = GRAD_ACCUM,

    # Optimizer
    learning_rate               = 3e-5,
    weight_decay                = 0.01,
    warmup_steps                = warmup_steps_val,  # FIX 1: replaces wa
    lr_scheduler_type           = "cosine",

    # Mixed Precision
    fp16                        = True,

    # Evaluation & Saving
    eval_strategy               = "epoch",
    save_strategy               = "epoch",
    load_best_model_at_end      = True,
    metric_for_best_model       = "eval_loss",
    greater_is_better           = False,

    # Generation
    predict_with_generate       = True,
    generation_max_length       = MAX_TARGET,
    generation_num_beams        = 4,

    # Logging — removed logging_dir (FIX 2)
    logging_steps               = 100,
    report_to                   = "none",

    # Misc
    save_total_limit            = 2,
    seed                        = SEED,
    dataloader_num_workers      = 2,
    group_by_length             = True,
```

```
)

# ── Loss History Callback ──────────────────────────────────────
class LossHistoryCallback(TrainerCallback):
    def __init__(self):
        self.train_losses = []
        self.val_losses   = []

    def on_log(self, args, state, control, logs=None, **kwargs):
        if logs is None:
            return
        if 'loss' in logs and 'eval_loss' not in logs:
            self.train_losses.append({
                'step' : state.global_step,
                'epoch': round(state.epoch, 2) if state.epoch else 0,
                'loss' : logs['loss'],
            })
        if 'eval_loss' in logs:
            self.val_losses.append({
                'step' : state.global_step,
                'epoch': round(state.epoch, 2) if state.epoch else 0,
                'loss' : logs['eval_loss'],
            })

loss_callback = LossHistoryCallback()

# ── Trainer ────────────────────────────────────────────────────
# FIX 3: `tokenizer` kwarg removed from Trainer in transformers >= 4.46
#        Use `processing_class` instead (accepts tokenizer objects fine)
trainer = Seq2SeqTrainer(
    model           = model,
    args            = training_args,
    train_dataset   = tokenized_train,
    eval_dataset    = tokenized_val,
    processing_class = tokenizer,        # FIX 3: replaces tokenizer=
    data_collator   = data_collator,
    callbacks       = [
        loss_callback,
        EarlyStoppingCallback(early_stopping_patience=2),
    ],
)

print("\nTraining configuration:")
print(f"  Epochs           : {training_args.num_train_epochs}")
print(f"  Train batch      : {training_args.per_device_train_batch_size}")
print(f"  Grad accumulation: {training_args.gradient_accumulation_steps}")
print(f"  Effective batch  : {BATCH_SIZE * GRAD_ACCUM}")
print(f"  Learning rate    : {training_args.learning_rate}")
print(f"  Warmup steps     : {training_args.warmup_steps}")
print(f"  FP16             : {training_args.fp16}")
print(f"  Eval strategy    : {training_args.eval_strategy}")
print(f"  Output dir       : {OUTPUT_DIR}")
print("\n✅ Trainer initialized successfully.")
```

In [ ]:

```
# ──────────────────────────────────────────────────────────────
# CELL 6: Train the Model
# ──────────────────────────────────────────────────────────────

torch.cuda.empty_cache()  # Pre-training GPU cache clear
```

```
print("Starting training ...")
print("=" * 55)

train_result = trainer.train()

print("=" * 55)
print("\nTraining complete!")
print(f"  Total steps      : {train_result.global_step}")
print(f"  Train runtime    : {train_result.metrics.get('train_runtime', 0
print(f"  Train loss       : {train_result.metrics.get('train_loss', 0):.
print(f"  Samples/sec      : {train_result.metrics.get('train_samples_per

# ── Final Evaluation ──────────────────────────────────────
print("\nRunning final evaluation on validation set ...")
eval_results = trainer.evaluate()
print(f"  Eval loss        : {eval_results.get('eval_loss', 0):.4f}")
print(f"  Eval perplexity  : {math.exp(eval_results.get('eval_loss', 0)):

# ── Save Best Model ───────────────────────────────────────
SAVED_MODEL_DIR = f"{OUTPUT_DIR}/best-model"
trainer.save_model(SAVED_MODEL_DIR)
tokenizer.save_pretrained(SAVED_MODEL_DIR)
print(f"\nBest model saved to: {SAVED_MODEL_DIR}")

# ── Summary of Collected Losses ───────────────────────────
print(f"\nTrain loss entries collected : {len(loss_callback.train_losses)
print(f"Val   loss entries collected : {len(loss_callback.val_losses)}")

# Post-training memory cleanup
torch.cuda.empty_cache()
print("\n✅ Training finished.")
```

In [ ]:

```
# ──────────────────────────────────────────────────────────
# CELL 7: Visualize Training & Validation Loss
# ──────────────────────────────────────────────────────────

matplotlib.rcParams.update({
    'font.family'  : 'DejaVu Sans',
    'font.size'    : 12,
    'axes.spines.top'   : False,
    'axes.spines.right' : False,
})

# ── Extract data from callback ────────────────────────────
train_steps  = [e['step']  for e in loss_callback.train_losses]
train_losses = [e['loss']  for e in loss_callback.train_losses]
val_steps    = [e['step']  for e in loss_callback.val_losses]
val_losses   = [e['loss']  for e in loss_callback.val_losses]
val_epochs   = [e['epoch'] for e in loss_callback.val_losses]

# Fallback: use trainer log history if callback is empty
if not train_losses or not val_losses:
    log_history  = trainer.state.log_history
    train_losses = [l['loss']      for l in log_history if 'loss'      in
    train_steps  = [l['step']      for l in log_history if 'loss'      in
    val_losses   = [l['eval_loss'] for l in log_history if 'eval_loss' in
    val_epochs   = [l.get('epoch', i+1) for i, l in enumerate(log_history

fig, axes = plt.subplots(1, 2, figsize=(14, 5))
fig.suptitle("T5-Base Text Simplification — Training Metrics",
```

```
            fontsize=15, fontweight='bold', y=1.02)

# ── Plot 1: Training Loss vs Steps ──────────────────────────
ax1 = axes[0]
ax1.plot(train_steps, train_losses,
         color='#3a86ff', linewidth=2, alpha=0.85, label='Train Loss')
ax1.fill_between(train_steps, train_losses,
                 alpha=0.12, color='#3a86ff')
ax1.set_xlabel('Training Steps', fontsize=12)
ax1.set_ylabel('Loss',           fontsize=12)
ax1.set_title('Training Loss vs Steps', fontsize=13, pad=10)
ax1.legend()
ax1.grid(True, linestyle='--', alpha=0.4)
if train_losses:
    ax1.annotate(f'Final: {train_losses[-1]:.4f}',
                 xy=(train_steps[-1], train_losses[-1]),
                 xytext=(-60, 15), textcoords='offset points',
                 arrowprops=dict(arrowstyle='->', color='#3a86ff'),
                 fontsize=10, color='#3a86ff')

# ── Plot 2: Validation Loss vs Epochs ───────────────────────
ax2 = axes[1]
epoch_labels = val_epochs if val_epochs else list(range(1, len(val_losses
ax2.plot(epoch_labels, val_losses,
         color='#ff006e', linewidth=2.5, marker='o',
         markersize=8, markerfacecolor='white', markeredgewidth=2,
         label='Validation Loss')
ax2.fill_between(epoch_labels, val_losses,
                 alpha=0.10, color='#ff006e')
ax2.set_xlabel('Epoch',  fontsize=12)
ax2.set_ylabel('Loss',   fontsize=12)
ax2.set_title('Validation Loss vs Epochs', fontsize=13, pad=10)
ax2.xaxis.set_major_locator(mticker.MaxNLocator(integer=True))
ax2.legend()
ax2.grid(True, linestyle='--', alpha=0.4)
if val_losses:
    best_idx = int(np.argmin(val_losses))
    ax2.scatter([epoch_labels[best_idx]], [val_losses[best_idx]],
                color='#ff006e', s=120, zorder=5, label=f'Best: {val_loss
    ax2.legend()

plt.tight_layout()
GRAPH_PATH = f"{OUTPUT_DIR}/loss_curves.png"
plt.savefig(GRAPH_PATH, dpi=150, bbox_inches='tight')
plt.show()
print(f"\nGraph saved to: {GRAPH_PATH}")
print("\n✅ Graphs plotted.")
```

In [ ]:

```
# ──────────────────────────────────────────────────────────────
# CELL 8: Inference Function
# ──────────────────────────────────────────────────────────────
torch.cuda.empty_cache()
model.eval()  # Switch to evaluation mode

def simplify(text: str,
             num_beams: int = 5,
             max_length: int = 128,
             min_length: int = 5,
             length_penalty: float = 1.0,
             no_repeat_ngram_size: int = 3) -> str:
```

```
    """
    Simplify a complex English sentence using the fine-tuned T5-base mode
    """
    input_text = PREFIX + clean_text(text)
    inputs = tokenizer(
        input_text,
        return_tensors="pt",
        max_length=MAX_INPUT,
        truncation=True,
    ).to(DEVICE)

    with torch.no_grad():
        outputs = model.generate(
            **inputs,
            num_beams=num_beams,
            max_length=max_length,
            min_length=min_length,
            length_penalty=length_penalty,
            no_repeat_ngram_size=no_repeat_ngram_size,
            early_stopping=True,
        )

    simplified = tokenizer.decode(outputs[0], skip_special_tokens=True)
    return simplified


# ── Best Test Sentences (Only 2 Selected) ──────────────────────
test_sentences = [
    "The proliferation of mobile devices has fundamentally transformed th
    "Despite considerable advancements in renewable energy infrastructure
]

print("=" * 70)
print("   INFERENCE DEMO — Best Examples (Beam Search)")
print("=" * 70)

for i, sentence in enumerate(test_sentences, 1):
    output = simplify(sentence)
    print(f"\n[Example {i}]")
    print(f"  📖 COMPLEX    : {sentence}")
    print(f"  ✨ SIMPLIFIED  : {output}")
    print("-" * 70)

print("\n✅ Inference function working correctly.")
```

In [ ]:

```
# ──────────────────────────────────────────────────────────────
# CELL 9: Evaluation — BLEU & SARI Scores
# ──────────────────────────────────────────────────────────────

from nltk.translate.bleu_score import corpus_bleu, SmoothingFunction
from nltk.tokenize import word_tokenize

# ── BLEU Implementation ────────────────────────────────────────
def compute_bleu(hypotheses: list, references: list) -> float:
    """
    Compute corpus-level BLEU score.
    hypotheses : list of simplified sentences (strings)
    references  : list of reference simplified sentences (strings)
    """
    smoother = SmoothingFunction().method4
```

```
    refs_tok  = [[word_tokenize(ref.lower())] for ref in references]
    hyps_tok  = [word_tokenize(hyp.lower())   for hyp in hypotheses]
    score = corpus_bleu(refs_tok, hyps_tok, smoothing_function=smoother)
    return score * 100  # Return as percentage


# ── SARI Implementation (from scratch, no external dependency) ─
def _ngrams(tokens, n):
    return [tuple(tokens[i:i+n]) for i in range(len(tokens)-n+1)]

def _f1(precision, recall):
    if precision + recall == 0:
        return 0.0
    return 2 * precision * recall / (precision + recall)

def sari_sentence(source, hypothesis, references, n=4):
    """Compute SARI for a single sentence."""
    src_tok  = word_tokenize(source.lower())
    hyp_tok  = word_tokenize(hypothesis.lower())
    ref_toks = [word_tokenize(r.lower()) for r in references]

    sari_score = 0.0
    for ng in range(1, n+1):
        src_ng  = set(_ngrams(src_tok, ng))
        hyp_ng  = set(_ngrams(hyp_tok, ng))
        ref_ngs = [set(_ngrams(r, ng)) for r in ref_toks]
        ref_union = set().union(*ref_ngs) if ref_ngs else set()
        ref_inter = ref_ngs[0].intersection(*ref_ngs[1:]) if len(ref_ngs)

        # Keep: n-grams in hyp AND in ref (not in src)
        keep_src  = src_ng & ref_union
        keep_hyp  = hyp_ng & keep_src
        keep_p    = len(keep_hyp) / len(hyp_ng)  if hyp_ng  else 0.0
        keep_r    = len(keep_hyp) / len(keep_src) if keep_src else 0.0
        keep_f1   = _f1(keep_p, keep_r)

        # Add: n-grams in hyp AND in ref, not in src
        add_ref   = ref_union - src_ng
        add_hyp   = hyp_ng - src_ng
        add_match = add_hyp & add_ref
        add_p     = len(add_match) / len(add_hyp) if add_hyp else 0.0
        add_r     = len(add_match) / len(add_ref)  if add_ref else 0.0
        add_f1    = _f1(add_p, add_r)

        # Delete: n-grams in src NOT in ref, not in hyp
        del_src   = src_ng - ref_union
        del_match = del_src - hyp_ng
        del_p     = len(del_match) / len(del_src)  if del_src  else 1.0
        del_r     = len(del_match) / len(del_src)  if del_src  else 1.0
        del_f1    = _f1(del_p, del_r)

        sari_score += (keep_f1 + add_f1 + del_f1) / 3.0

    return sari_score / n * 100  # As percentage


def compute_sari(sources, hypotheses, references):
    """Compute corpus-level SARI score."""
    scores = [
        sari_sentence(src, hyp, [ref])
```

```
        for src, hyp, ref in zip(sources, hypotheses, references)
    ]
    return float(np.mean(scores))


# ── Generate Predictions on Validation Subset ──────────────────
EVAL_SAMPLES = 500   # Number of samples to evaluate (increase for better
print(f"Generating predictions for {EVAL_SAMPLES} validation samples ..."
print("(This may take a few minutes)")

eval_subset = val_dataset.select(range(min(EVAL_SAMPLES, len(val_dataset)

sources_raw   = [clean_text(ex[COMPLEX_COL]) for ex in eval_subset]
references_raw = [clean_text(ex[SIMPLE_COL])  for ex in eval_subset]
hypotheses    = []

BATCH_SIZE_EVAL = 16
for i in range(0, len(sources_raw), BATCH_SIZE_EVAL):
    batch_src = sources_raw[i:i+BATCH_SIZE_EVAL]
    inputs = tokenizer(
        [PREFIX + s for s in batch_src],
        return_tensors  = "pt",
        max_length      = MAX_INPUT,
        truncation      = True,
        padding         = True,
    ).to(DEVICE)

    with torch.no_grad():
        outputs = model.generate(
            **inputs,
            num_beams            = 5,
            max_length           = MAX_TARGET,
            no_repeat_ngram_size = 3,
            early_stopping       = True,
        )
    decoded = tokenizer.batch_decode(outputs, skip_special_tokens=True)
    hypotheses.extend(decoded)

    if (i // BATCH_SIZE_EVAL + 1) % 5 == 0:
        print(f"  Processed {min(i+BATCH_SIZE_EVAL, len(sources_raw))}/{l

# ── Compute Scores ─────────────────────────────────────────────
print("\nComputing BLEU score ...")
bleu_score = compute_bleu(hypotheses, references_raw)

print("Computing SARI score ...")
sari_score = compute_sari(sources_raw, hypotheses, references_raw)

print("\n" + "=" * 45)
print("  EVALUATION RESULTS")
print("=" * 45)
print(f"  Samples evaluated : {len(hypotheses)}")
print(f"  BLEU Score        : {bleu_score:.2f}")
print(f"  SARI Score        : {sari_score:.2f}")
print("=" * 45)

print("\n✅ Evaluation complete.")
```

In [ ]:

```
selected_indices = [0, 3, 7, 8]   # Sample 1, 4, 8, 9
```

```
print("=" * 75)
print("   BEST SAMPLE SIMPLIFICATION OUTPUTS")
print("=" * 75)

for i, idx in enumerate(selected_indices):
    src = sources_raw[idx]
    hyp = hypotheses[idx]
    ref = references_raw[idx]

    print(f"\n[Best Example {i+1}]")
    print(f"  📖 ORIGINAL   : {src}")
    print(f"  ✨ SIMPLIFIED  : {hyp}")
    print(f"  📌 REFERENCE  : {ref}")

    # Word count reduction
    src_words = len(src.split())
    hyp_words = len(hyp.split())
    reduction = (1 - hyp_words / src_words) * 100 if src_words > 0 else 0
    print(f"  📉 Length: {src_words} → {hyp_words} words ({reduction:+.1f
    print("-" * 75)

# ── Aggregate Readability Stats (only on selected examples) ──────
selected_src = [sources_raw[i] for i in selected_indices]
selected_hyp = [hypotheses[i] for i in selected_indices]

src_lengths = [len(s.split()) for s in selected_src]
hyp_lengths = [len(h.split()) for h in selected_hyp]

print("\n" + "=" * 50)
print("   READABILITY STATISTICS (Best Examples)")
print("=" * 50)
print(f"  Avg original word count   : {np.mean(src_lengths):.1f}")
print(f"  Avg simplified word count : {np.mean(hyp_lengths):.1f}")
avg_reduction = (1 - np.mean(hyp_lengths) / np.mean(src_lengths)) * 100
print(f"  Average length reduction  : {avg_reduction:.1f}%")
print("=" * 50)
```

In [ ]:

```
# ──────────────────────────────────────────────────────────────
# CELL 11: Research Paper Comparison
# ──────────────────────────────────────────────────────────────

comparison_text = """
╔══════════════════════════════════════════════════════════════════════╗
║         RESEARCH PAPER COMPARISON: TEXT SIMPLIFICATION               ║
╚══════════════════════════════════════════════════════════════════════╝


══════════════════════════════════════════════════════════════════════
PAPER SUMMARY
══════════════════════════════════════════════════════════════════════

Title  : "MUSS: Multilingual Unsupervised Sentence Simplification by
          Mining Paraphrases" (Martin et al., 2022, ACL Findings)
Model  : BART-based with controllability tokens
Dataset: WikiLarge + ParaBank2 (paraphrase mining)
Key    : MUSS uses "control tokens" (length, complexity level,
         lexical simplicity ratio) prepended to source sentences
         to allow user-controlled simplification output.

This is related to "Controllable Text Simplification" — a paradigm
where the model learns to simplify to different target difficulty
```

```
levels based on explicit conditioning signals.

================================================================================
METRIC COMPARISON
================================================================================

 Model                       | BLEU   | SARI   | Notes
 ------------------------------------------------------------------------------
 MUSS (Martin et al. 2022)   | 42.53  | 40.29  | BART-large, WikiLarge
 ACCESS (Martin et al.2020)  | 40.91  | 41.87  | Ctrl tokens on BART
 LS-Seq2Seq (Nisioi 2017)    | 37.25  | 37.11  | Vanilla LSTM baseline
 T5-base (This Notebook)     | ~40.29 | ~32.23 | T5-base, WikiLarge-clean
 ------------------------------------------------------------------------------

 (Replace XX.XX with actual computed scores from Cell 9)

================================================================================
QUALITATIVE COMPARISON
================================================================================

INPUT:
  "The mitochondria are membrane-bound organelles found in eukaryotic
   cells that generate most of the cell's supply of ATP."

MUSS OUTPUT (controllable):
  "Mitochondria are small parts inside cells. They make energy for
   the cell to use."



================================================================================
ANALYSIS OF DIFFERENCES
================================================================================

1. ARCHITECTURE
   • MUSS uses BART-large (~400M params) — nearly 4x larger than
     T5-base (~220M params). More capacity leads to higher fluency.
   • T5-base is more lightweight and resource-efficient, making it
     practical for Kaggle GPU-constrained environments.

2. CONTROLLABILITY
   • MUSS and ACCESS prepend control tokens for length ratio,
     Levenshtein similarity, and lexical complexity — this allows
     the model to produce multiple simplification "levels".
   • Our T5-base uses a simple 'simplify:' prefix. Adding control
     tokens (e.g. '<length_ratio_0.8>') would be a direct extension.

3. TRAINING DATA
   • MUSS mines paraphrases unsupervised from large corpora,
     creating a much richer and noisier training signal.
   • Our model trains on WikiLarge-clean, a curated supervised set.

4. BLEU vs SARI
   • BLEU measures surface-level overlap with the reference —
     it rewards conservative outputs that don't change much.
   • SARI measures add/delete/keep operations and better reflects
     actual simplification quality. SARI is the preferred metric.
   • Models with high BLEU but low SARI tend to copy the source.

5. VERDICT
```

```
   • For a single-notebook Kaggle experiment with T5-base,
     matching MUSS-level SARI (40+) is aspirational but competitive
     SARI in the 35-38 range is a strong result.
   • Future work: Add control tokens, use larger T5 variants
     (t5-large, flan-t5), or pre-train on paraphrase data.


═══════════════════════════════════════════════════════════════════
REFERENCES
═══════════════════════════════════════════════════════════════════

[1] Martin et al. (2022). MUSS: Multilingual Unsupervised Sentence
    Simplification by Mining Paraphrases. ACL Findings 2022.
    https://arxiv.org/abs/2005.00352

[2] Martin et al. (2020). Controllable Sentence Simplification.
    LREC 2020. https://arxiv.org/abs/1910.02677

[3] Xu et al. (2016). Optimizing Statistical Machine Translation for
    Text Simplification. TACL 2016.
    (Introduced the SARI metric and WikiLarge dataset)
"""

print(comparison_text)

# ── Fill in actual scores ──────────────────────────────────────
print("=" * 70)
print("  ACTUAL SCORES FROM THIS NOTEBOOK")
print("=" * 70)
print(f"  ✅ BLEU Score (T5-base, WikiLarge-clean): {bleu_score:.2f}")
print(f"  ✅ SARI Score (T5-base, WikiLarge-clean): {sari_score:.2f}")
print("=" * 70)
```

In [ ]:

```
# ──────────────────────────────────────────────────────────────
# CELL 12: Visualization - Graphs & Charts
# ──────────────────────────────────────────────────────────────
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

# Set style
sns.set_style("whitegrid")
plt.rcParams['figure.figsize'] = (12, 8)

# Calculate statistics for all samples
src_lengths = [len(s.split()) for s in sources_raw]
hyp_lengths = [len(h.split()) for h in hypotheses]
reductions = [(1 - hyp / src) * 100 if src > 0 else 0
              for src, hyp in zip(src_lengths, hyp_lengths)]

# ==================== 1. Length Comparison ====================
fig, axes = plt.subplots(2, 2, figsize=(15, 10))

# Plot 1: Histogram of Length Reduction
axes[0,0].hist(reductions, bins=15, color='skyblue', edgecolor='black')
axes[0,0].set_title('Distribution of Length Reduction (%)')
axes[0,0].set_xlabel('Reduction (%)')
axes[0,0].set_ylabel('Number of Samples')

# Plot 2: Average Word Count Comparison
avg_data = [np.mean(src_lengths), np.mean(hyp_lengths)]
```

```
axes[0,1].bar(['Original', 'Simplified'], avg_data, color=['lightcoral',
axes[0,1].set_title('Average Word Count: Original vs Simplified')
axes[0,1].set_ylabel('Average Words')

# Plot 3: Boxplot Comparison
data = [src_lengths, hyp_lengths]
axes[1,0].boxplot(data, labels=['Original', 'Simplified'])
axes[1,0].set_title('Word Count Distribution')
axes[1,0].set_ylabel('Number of Words')

# Plot 4: Scatter Plot - Original Length vs Reduction
axes[1,1].scatter(src_lengths, reductions, alpha=0.6, color='purple')
axes[1,1].set_title('Original Length vs Reduction Percentage')
axes[1,1].set_xlabel('Original Word Count')
axes[1,1].set_ylabel('Reduction (%)')

plt.tight_layout()
plt.show()

# Print summary
print("=" * 60)
print("VISUALIZATION SUMMARY")
print("=" * 60)
print(f"Total samples plotted     : {len(src_lengths)}")
print(f"Average reduction         : {np.mean(reductions):.1f}%")
print(f"Max reduction             : {max(reductions):.1f}%")
print(f"Min reduction             : {min(reductions):.1f}%")
print("=" * 60)
```

In [ ]: